

Ambulance Base Placement for Emergency Response Optimisation

Deepak Saai - BT2024138

Kaushiksai Yakkala - BT2024240

Saravanan S B - BT2024254

Problem

Emergency medical response systems rely on rapid ambulance deployment to save lives, particularly in Tier-2 Indian cities where infrastructure and resource allocation are often suboptimal. Inefficient placement of ambulance bases results in long travel distances, uneven citywide coverage, and delays that can be the difference between life and death.

The central question is:

Where should ambulance bases be located, and which zones should they serve, to minimize emergency response time under practical constraints?

In this project, we model this planning challenge using real road-network travel times extracted from OpenStreetMap and represent the city as a set of demand zones. Through optimization techniques, we determine the best locations for ambulance bases and analyze how performance improves as the number of allowed bases increases. This problem closely mirrors real decisions made by urban health authorities, making the study both applicable and impactful.

Dataset Overview

The analysis uses openly available geographic and network data from OpenStreetMap, accessed through the OSMnx Python library.

- The city is divided into a grid of rectangular zones, each representing a region of emergency demand.
- The centroids of these zones are used as representative points for computing travel times.

- A set of candidate locations for ambulance bases is generated from network nodes.
- All zones are treated equally in terms of demand weight.

Assumptions

- Travel times are static (do not change by time of day).
- Ambulances respond from the nearest open base only.
- Each zone must be assigned to exactly one base.
- Demand per zone is uniform unless weights are provided.
- Opening a base has a fixed cost.
- All bases have equal capability.

Model Choice

To solve the base-location problem, we use the **p-median facility location model**, a widely used optimization framework for minimizing demand-weighted travel distances.

- **Objective:** Minimize weighted travel time between zones and their corresponding bases.
- **Decision variables:** Binary (0/1) for assignments and base openings.
- **Reason:** Ensures valid assignments, supports budget constraints, and handles cost–benefit trade-offs.

Objective Function

$$\min \sum_i \sum_j w_i \cdot T_{ij} \cdot x_{ij}$$

Where:

- w_i = demand weight at zone i
- T_{ij} = travel time from base j to zone i
- $x_{ij} = 1$ if zone i is assigned to base j

We additionally use a penalized objective:

$$J(K) = \text{AvgResponse}(K) + \lambda \times \text{TotalCost}(K)$$

We then selected the value of K for which $J(K)$ attains its minimum over the tested set of K values.

Constraints

Assignment Constraint

Each zone is assigned to exactly one base:

$$\sum_j x_{ij} = 1 \quad \forall i$$

Base-Open Constraint

A zone can only be assigned to an open base:

$$x_{ij} \leq y_j \quad \forall i, j$$

Budget Constraint

$$\sum_j y_j \leq K$$

Binary Variables

$$y_j \in \{0, 1\}, \quad x_{ij} \in \{0, 1\}$$

Solver / Optimizer

- Implemented in Python using PuLP.
- MILP solved using CBC (open-source).
- Suitable for small-to-medium problem sizes.
- Experiments run for $K = 1$ through $K = M$ (number of candidate bases).

1 Findings and Results

We first constructed a set of demand zones covering Mysore and removed zones whose polygons did not intersect the municipal boundary, which left 23 valid zones. Using OpenRoute-Service, we then built a travel time matrix T_{ij} in minutes between each zone centroid i and each candidate hospital base j .

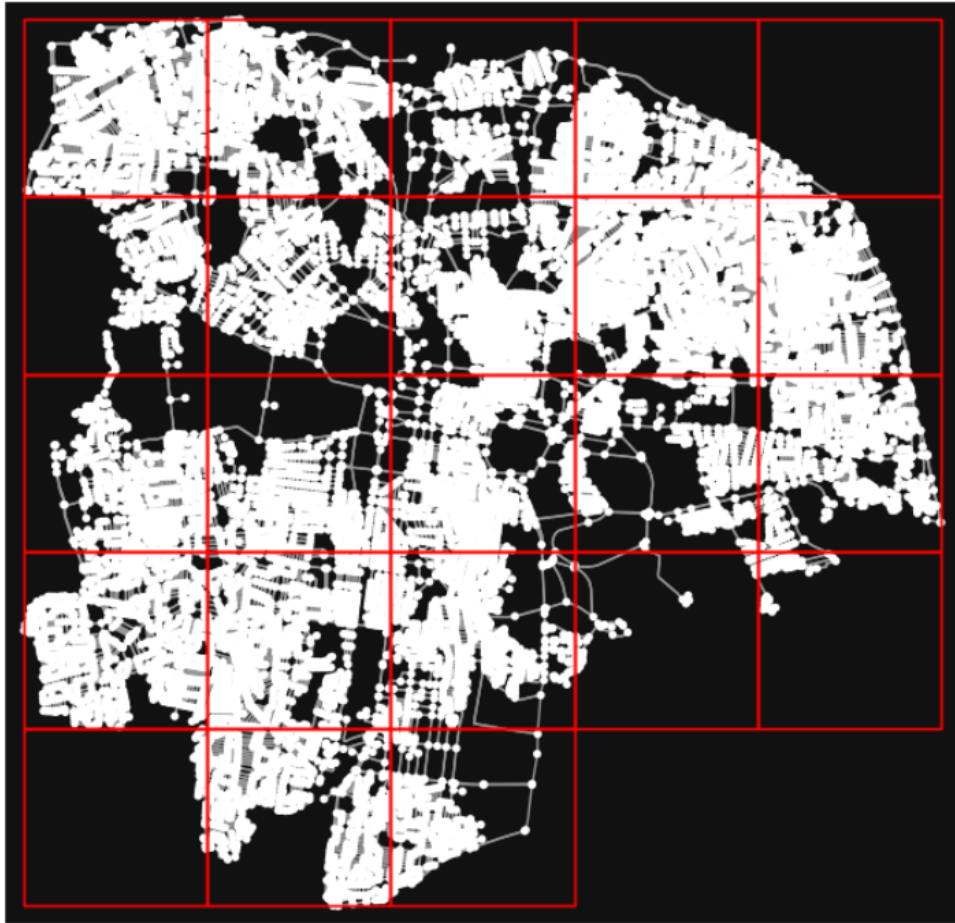


Figure 1: Demand zones and selected ambulance bases in Mysore.

For each allowed number of bases K , we solved a p median style MILP and recorded the open bases, zone assignments, and the resulting demand weighted average and maximum response times. As K increases, average response time decreases, but with clear diminishing returns: the biggest improvements come when moving from very few bases to a moderate number, after which extra bases only reduce time slightly.

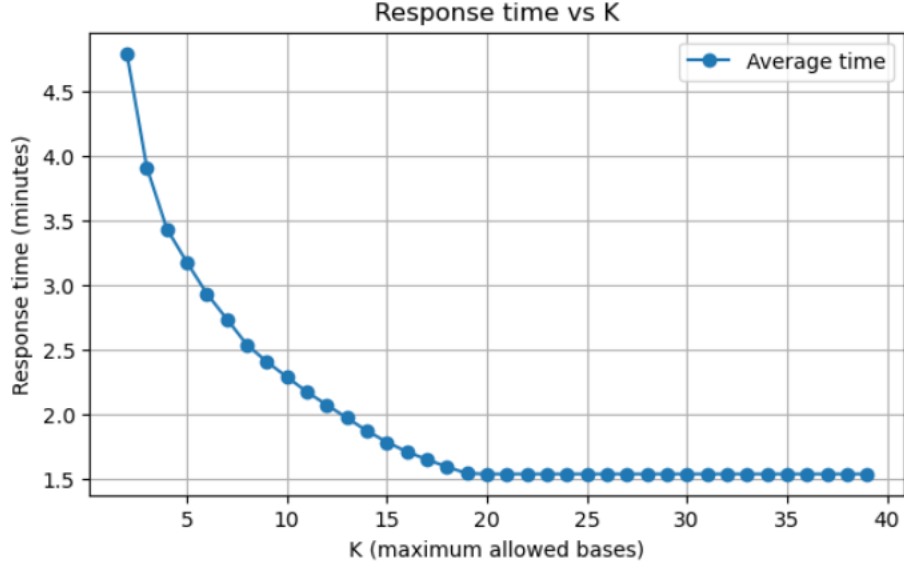


Figure 2: Diminishing Returns seen once reaching $K = 20$

We also attached a simple cost model where each base has a fixed cost and defined a penalized objective

$$J(K) = \text{avg_time}(K) + \lambda \cdot \text{cost}(K)$$

with $\lambda = 0.1$ minutes per crore. Plotting $J(K)$ across the tested values of K gave a clear minimum, which we used to select a “sweet spot” configuration that balances response time and infrastructure cost.

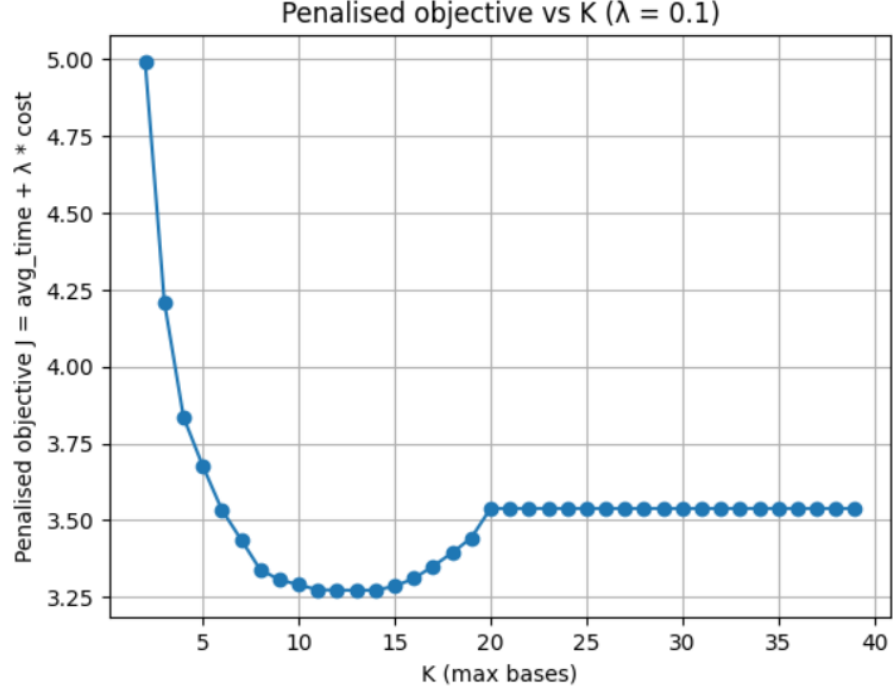


Figure 3: Time and Cost Trade Off once Imposing the Penalty.

For this value of λ , the penalized objective is minimized at $K = 13$, which we treat as the recommended number of bases. We then plot these 13 optimal bases and their zone assignments on the Mysore map to visualize the final deployment configuration.

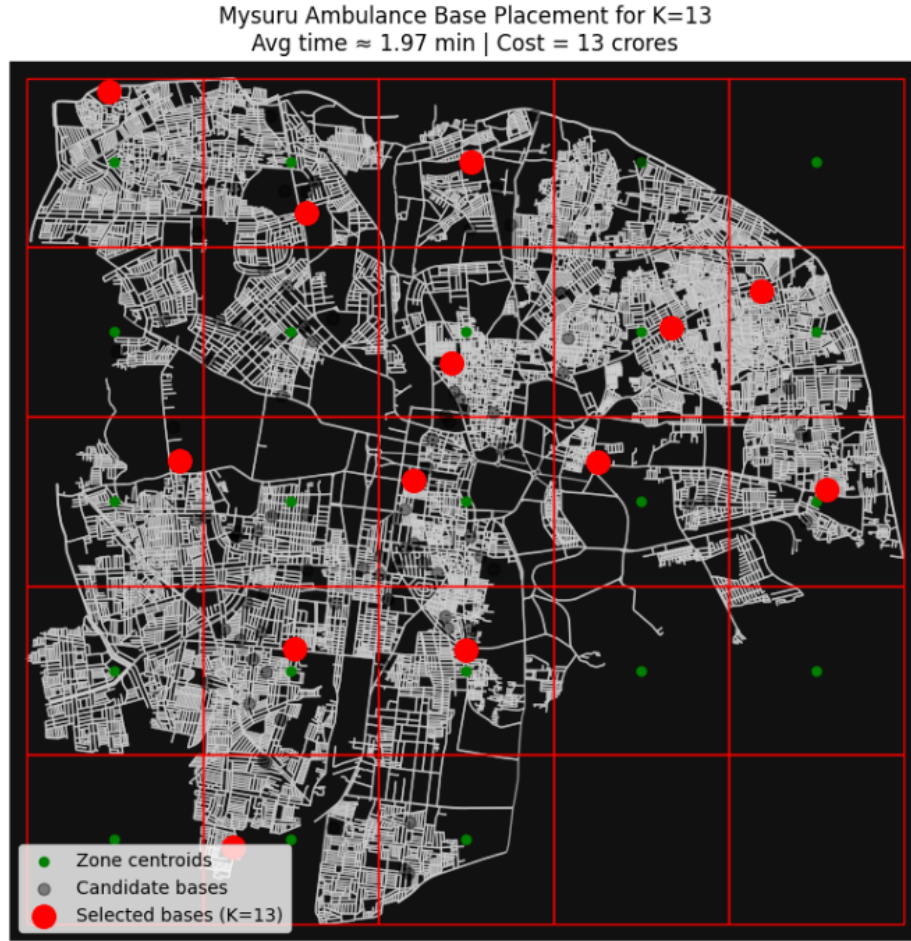


Figure 4: Plotting the Optimal Bases for $K = 13$.

Challenges & Fixes

- We were not able to compute accurate travel times with OSMnx. We used OpenRouteService which uses the actual road network and a driving profile to calculate more accurate travel times.
- Initially wanted to solve this issue for a bigger city but due to large number of candidate bases had to settle for a smaller city.
- Trimmed unrealistic boundary zones using city polygon.
- We calculated demand weights based on road network density as we could not calculate weights based on real-time traffic congestion.
- Removed disconnected graph components.

Future Improvements

- Incorporate real ambulance call data to get demand weights.
- The existing travel time matrix essentially assumes that road speeds remain constant throughout the day because it was constructed using a single routing profile. In actuality, morning and evening traffic can greatly lengthen travel times. Next, you could use real-time traffic congestion data or create distinct time matrices for various times, such as morning peak, evening peak, and off peak.
- Multiple ambulances per base + dynamic dispatch simulation.
- The open source CBC solver is adequate for the problem's current scale. The mixed integer program may become more difficult to solve in a reasonable amount of time if the city is divided into many more zones or if more constraints and objectives are added. If so, you might be able to handle larger networks by using commercial solvers like Gurobi.
- Equity constraints (max response time).
- Provide a straightforward dashboard that allows planners to select K or λ , view base locations on a map, and quickly access summary metrics like average time and total cost.
- Implementing metaheuristic methods for large-scale versions.